4주차 풀이 읽기자료

해설 사용 순서

먼저 학생 실습에서 문제를 독립적으로 풉니다. 이 자료는 힌트와 완전 해설을 포함합니다. 필요한 단계까지만 읽고 다시 풀이합니다.

표시된 예시는 강의에서 만든 예시입니다. 원문과 전체 제약은 각 공식 링크에서 확인합니다.

학생 실습

교사용 진행안

코드 전체 내려받기

104 · 단계별 힌트

LeetCode 104: Maximum Depth of Binary Tree · Easy

H1. 빈 트리 함수의 반환값은 무엇인가요?

H2. 왼쪽 방문 뒤 elif로 오른쪽을 생략해도 되나요?

H3. 작은 트리에서 depth(None)=0, depth(node)=1+max(left,right)를 도출합니다. 실행 해설은 (node,depth)의 명시적 스택도 제공합니다.

104 · 정당성과 비용

재귀는 자식 높이가 맞다고 가정하면 현재 높이를 정확히 결합합니다. 명시적 스택은 각 노드의 실제 root 깊이를 전달하며 최대를 기록하므로 같은 답입니다.

n=노드 수,h=높이. O(n) 시간, DFS 보조 O(h), 최악 O(n).

104 · 반례와 검증

Python 재귀 제한은 10^4 편향 트리보다 작을 수 있습니다. 작은 예제용 재귀 함수와 전체 제약용 반복 구현을 구분합니다.

```
root=[4,2,7,None,3] → 3
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

104 · 구현

아래 구현은 코드 묶음에서도 확인할 수 있습니다.

```
def max_depth(root):
    """Explicit stack supports skew trees within the official size limit."""
    if root is None:
        return 0
    stack, best = [(root, 1)], 0
    while stack:
        node, depth = stack.pop()
        best = max(best, depth)
        if node.left:
            stack.append((node.left, depth + 1))
        if node.right:
            stack.append((node.right, depth + 1))
    return best
```

102 · 단계별 힌트

LeetCode 102: Binary Tree Level Order Traversal · Medium

H1. 한 레벨을 처리하기 전 큐 길이는 얼마인가요?

H2. 자식을 추가한 뒤의 길이를 반복 조건으로 쓰면 어떻게 되나요?

H3. deque에서 시작 시점의 len(queue)개만 꺼내고 자식을 다음 레벨로 추가합니다.

102 · 정당성과 비용

각 외부 반복 시작 때 큐에는 현재 레벨만 있습니다. 그 수만 꺼내면 새 자식은 다음 레벨 큐로 남습니다. 왼쪽 자식부터 추가하면 순서도 유지됩니다.

n=노드 수,w=최대 폭. O(n) 시간, 큐 O(w), 출력 O(n).

102 · 반례와 검증

list.pop(0)는 남은 원소를 이동시킵니다. 넓은 트리에서는 반복하면 $O(n^2)$까지 갈 수 있습니다.

`root=[4,2,7,None,3] → [[4],[2,7],[3]]`

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

102 · 구현

아래 구현은 코드 묶음에서도 확인할 수 있습니다.

```
def level_order(root):
    if root is None:
        return []
    queue, result = deque([root]), []
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

98 · 단계별 힌트

LeetCode 98: Validate Binary Search Tree · Medium

H1. 6은 부모 10뿐 아니라 어느 조상과 비교해야 하나요?

H2. 실패한 자식의 False를 어떻게 전달하나요?

H3. 각 노드에 열린 범위 (lower,upper)를 전달합니다. 왼쪽 upper=node.val, 오른쪽 lower=node.val입니다.

98 · 정당성과 비용

범위는 모든 조상에서 온 제약의 교집합입니다. 범위를 만족하면 조상 관계를 모두 만족하고, 자식 범위를 줄여 같은 성질을 유지합니다. 한 위반이라도 전체가 False입니다.

n=노드 수,h=높이. O(n) 시간, 명시적 DFS 스택 O(h), 최악 O(n).

98 · 반례와 검증

예시의 6은 10보다 작지만 root 8의 오른쪽에 있으므로 실패합니다. 부모만 비교하는 검사는 이를 놓칩니다.

```
root=[8,3,10,None,None,6,12] → False
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

98 · 구현

아래 구현은 코드 묶음에서도 확인할 수 있습니다.

```
def valid_bst(root):
    stack = [(root, float("-inf"), float("inf"))]
    while stack:
        node, lower, upper = stack.pop()
        if node is None:
            continue
        if not lower < node.val < upper:
            return False
        stack.append((node.right, node.val, upper))
        stack.append((node.left, lower, node.val))
    return True
```

230 · 단계별 힌트

LeetCode 230: Kth Smallest Element in a BST · Medium

H1. 왼쪽·현재·오른쪽 순서는 어떤 값 순서인가요?

H2. k는 방문 수인가요, 배열 인덱스인가요?

H3. 명시적 스택으로 inorder 순회하고 노드를 꺼낼 때 k를 하나 줄입니다.

230 · 정당성과 비용

BST의 왼쪽 전체는 현재보다 작고 오른쪽 전체는 큽니다. 귀납적으로 inorder는 정렬 순서이므로 k번째 방문 값이 답입니다.

n=노드 수,h=높이. O(h+k) 시간(최악 O(n)), O(h) 보조 공간.

230 · 반례와 검증

inorder 배열을 만들었다면 k-1로 접근합니다. 배열 전체 생성은 시간·출력 아닌 보조 공간 O(n)입니다.

```
root=[5,2,8,1,3], k=3 → 3
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

230 · 구현

아래 구현은 코드 묶음에서도 확인할 수 있습니다.

```
def kth_smallest(root, k):
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        k -= 1
        if k == 0:
            return node.val
        node = node.right
    raise ValueError("k exceeds tree size")
```

실행과 참고자료

```
cd code
python demo_07.py
python demo_08.py
python verify.py
```

공통 객체와 표준 라이브러리 import는 `structures.py`, `solutions.py`에 있습니다. LeetCode에서는 공식 메서드 서명에 맞춰 함수를 연결합니다.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

재귀 깊이 식의 유도

없는 자식의 높이는 0입니다. 비어 있지 않은 노드에서 시작하는 경로는 현재 노드와 한쪽 자식의 경로로 구성되므로 높이는 자식 높이의 최댓값에 1을 더한 값입니다. 부분 트리 크기에 대한 귀납법으로 설명할 수 있습니다. 두 자식은 더 작은 부분 트리이고, 빈 트리가 기저 사례입니다.

`[4,2,7,None,3]`에서 노드 3과 7은 1, 노드 2는 2, 노드 4는 3을 반환합니다. 재귀 예제의 호출 스택은 O(h) 공간입니다. 반복 구현은 방문할 노드를 명시적으로 저장하므로 Python 재귀 제한을 높이지 않고 실행합니다.

중간 점검의 답과 근거

1. `[4,2,7,None,3]`: 깊이 3, 레벨 `[[4],[2,7],[3]]`. 경로 4,2,3의 노드 수는 3입니다.
2. `[8,3,10,None,None,6,12]`: 잘못된 BST입니다. 6은 부모 10보다 작지만 루트 8의 오른쪽 부분 트리에 있으므로 8보다 커야 합니다.
3. BST `[5,2,8,1,3]`: k=1,3,5의 답은 1,3,8입니다. 중위 순서는 `1,2,3,5,8`입니다.

빈 트리의 깊이는 0, 레벨 목록은 `[]`입니다. 노드 하나의 깊이는 1입니다. 엄격한 부등식을 등호 포함으로 바꾸면 중복 BST 값을 잘못 허용합니다.

실행 코드 읽기

`structures.py`는 `TreeNode`와 `tree(values)`를 정의합니다. 변환 함수는 레벨 순서의 값을 읽으며 `None`을 없는 자식으로 처리합니다. 입력 노드 생성 공간은 각 알고리즘의 보조 공간과 별도로 계산합니다.

`solutions.py`는 `deque`를 가져옵니다. 네 반복 풀이는 입력 노드와 연결을 보존합니다. `depth_recursive`는 작은 트리의 설명용입니다. LeetCode에서는 제공된 `TreeNode`를 사용하고 `class Solution` 안의 `maxDepth`, `levelOrder`, `isValidBST`, `kthSmallest` 메서드로 연결합니다.

`verify.py --student`는 직접 구현한 코드에 같은 검사를 적용합니다. 기준 풀이와 비교하는 로컬 검사는 디버깅에 사용하며, 앞의 정당성 설명을 대신하지 않습니다.